TP Systèmes Multitâches

T2 - Systèmes Temps-Réel et Sûreté de Fonctionnement

Bastien HUBERT

Master SETI - novembre 2022

1 Préambule

1 - 4) Le fichier `preambule.c` manipule diverses variables relatives au partage de ressources, parmi lesquelles :

- Un thread `thread_1`
- Un pointeur `semaphore` vers un sémaphore `/preambule_sem`
- Un mutex `mutex`

5 - 8) Le thread est créé a l'aide de `pthread_create(&thread_1, NULL, produce, NULL)`, et son point d'entrée est `void *produce(void *params)`. L'attente de la fin du thread se fait grâce à `pthread_join(thread_1, NULL)`. Enfin, le thread par défaut du processus courant est celui qui lance `int main()`.

9) La figure 1 correspond au diagramme expliquant la conception détaillée de ce processus et de ce thread :

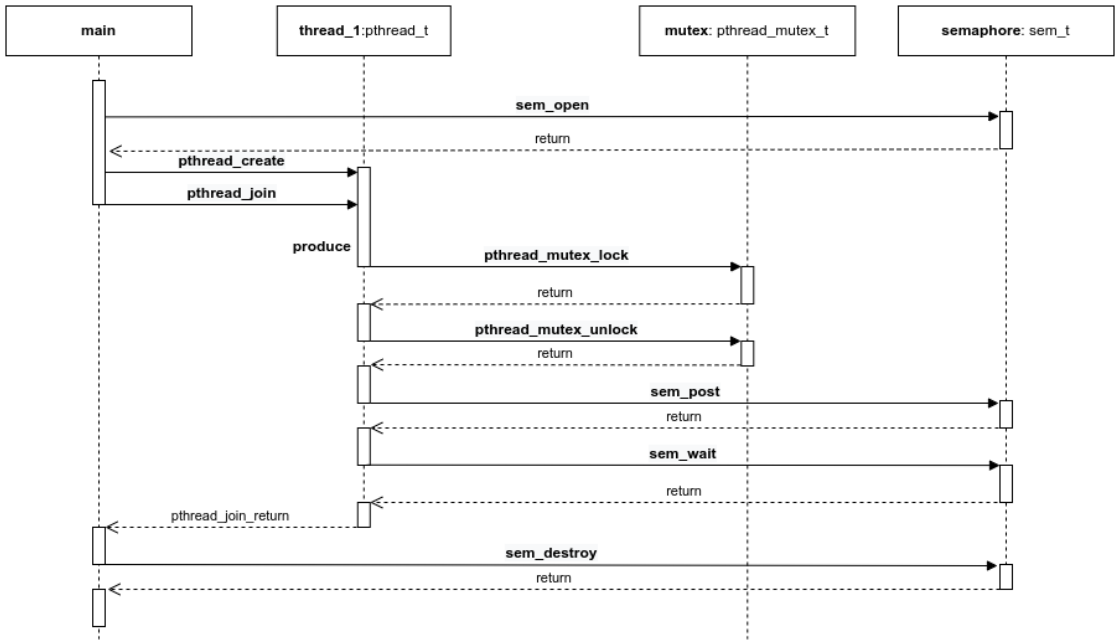


Figure 1: Conception détaillée de `preambule.c`

10) Ce diagramme ne prend pas en compte la gestion des erreurs, sa conception n'est donc pas complète.

2 Premier exercice (Exercice Principal)

On rappelle que les exigences assignées à `MultitaskingAccumulator` sont les suivantes :

- Exigence 1 : `MultitaskingAccumulator` doit acquérir quatre entrées et produire en sortie le cumul des entrées dès qu'une entrée est acquise. Le cumul correspond à la production d'un tableau de 256 entiers dont le ieme élément de ce tableau est la somme des iemes éléments des tableaux d'entrée.

- Exigence 2 : `MultitaskingAccumulator` doit acquérir toutes les données d'entrées.
- Exigence 3 : `MultitaskingAccumulator` doit garantir que les données d'entrée sont correctement formées avant de les sommer.
- Exigence 4 : `MultitaskingAccumulator` doit sommer et produire une sortie au plus vite, c'est-à-dire dès qu'une entrée est présente sans attendre une nouvelle valeur sur chacune des entrées.
- Exigence 5 : Le logiciel doit produire sur la sortie diagnostic pour chaque opération de cumul, combien d'entrées ont été acquises, combien ont été sommées et combien restent à sommer.

1) La figure 2 représente la répartition des exigences de la spécification sur les composants de `MultitaskingAccumulator` :

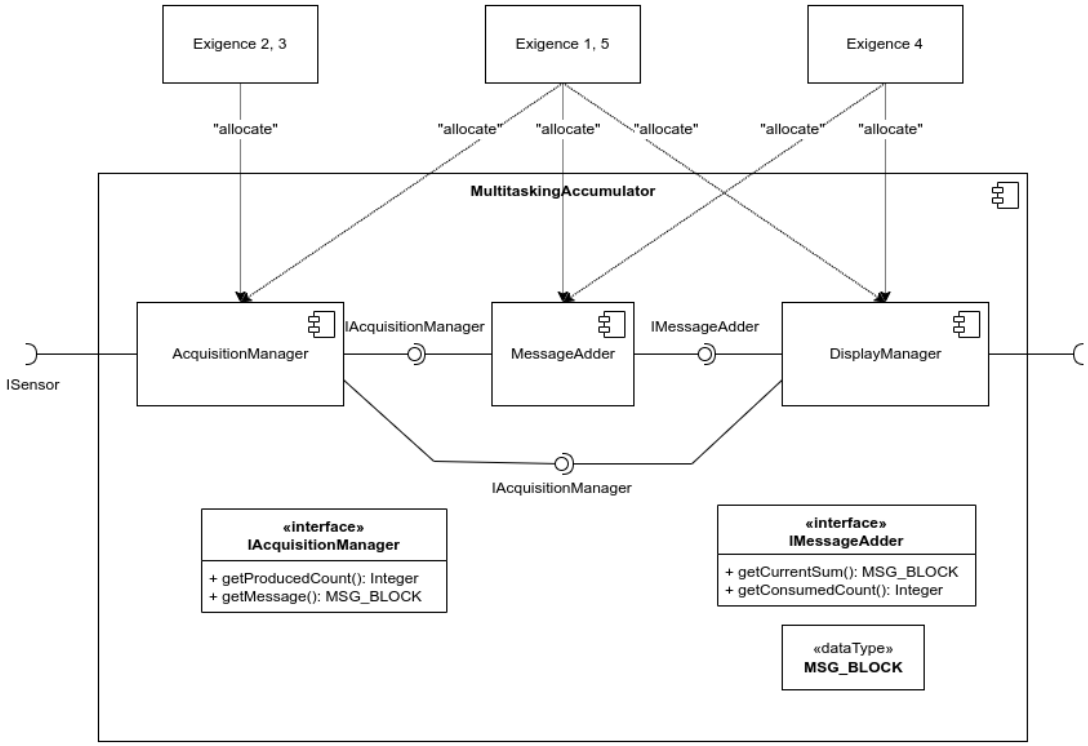


Figure 2: Architecture logicielle avec allocation des exigences

2) Une approche dirigée par les évènements est une approche asynchrone car on ne peut pas prédire à l'avance le moment où les évènements vont se produire.

3) `messageCheck` correspond à de la détection de faute, plus particulièrement à un *sanity check*. En effet, cette fonction effectue un test simple visant à vérifier que le contenu du message n'est pas immédiatement faux et donc corrompu.

4) À ce stade de l'implémentation, il y a 1 processus et 7 fils d'exécution (le thread par défaut, 3 pour `AcquisitionManager`, 1 pour `MessageAdder`, et 1 pour `DisplayManager`).

5) La figure 3 présente en détails l'architecture logicielles de `MultitaskingAccumulator` :

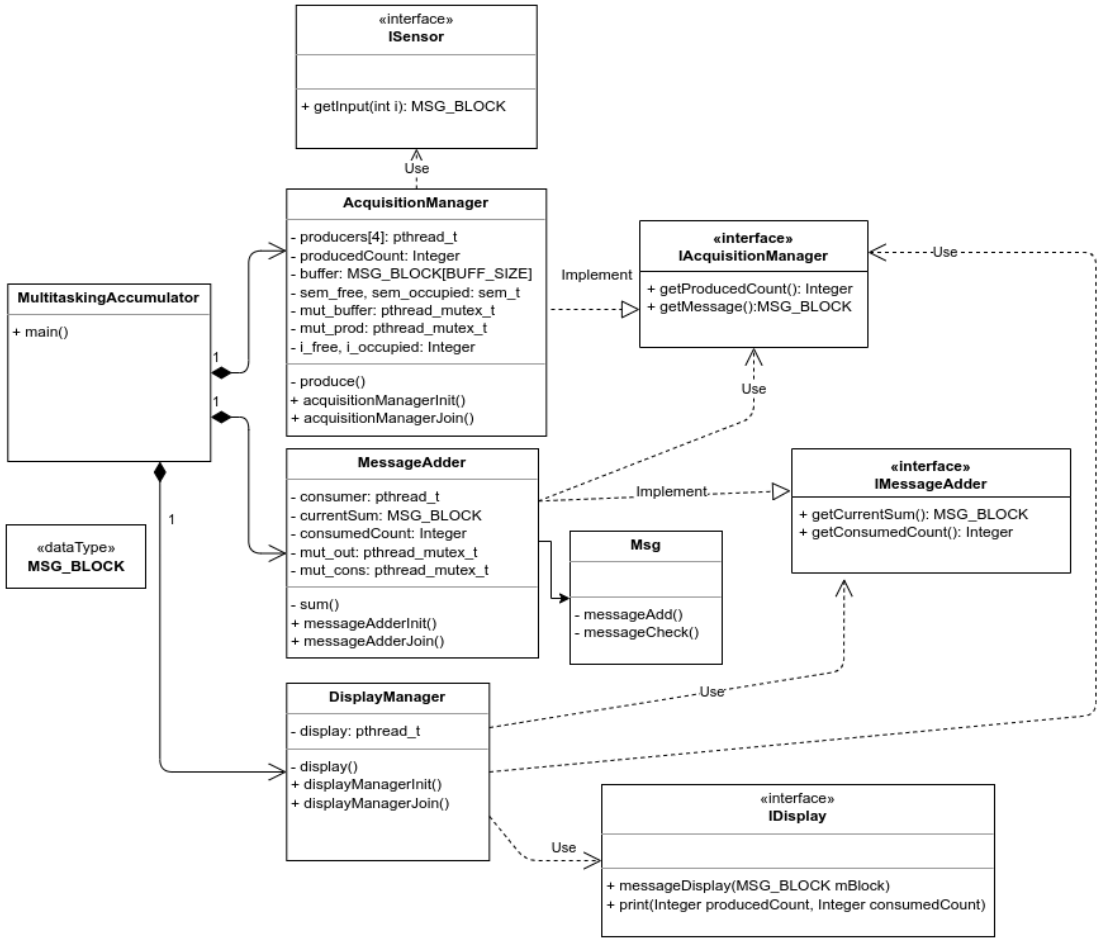


Figure 3: Architecture logicielle avec allocation des exigences

La figure 4 correspond à un diagramme séquentiel de `MultitaskingAccumulator` :

6) Les codes correspondants à l'implémentation de cette architecture sont joins avec ce rapport. Son exécution conduit à la sortie suivante :

```
[multitaskingAccumulator]Software initialization in progress...
[acquisitionManager]Synchronization initialization in progress...
[acquisitionManager]Creating semaphores
[acquisitionManager]Semaphores created
[acquisitionManager]Synchronization initialization done.
[acquisitionManager]Creating threads
[acquisitionManager]Thread created for produce with id 59585
[acquisitionManager]Thread created for produce with id 59586
[acquisitionManager]Threads created
[acquisitionManager]Thread created for produce with id 59587
```

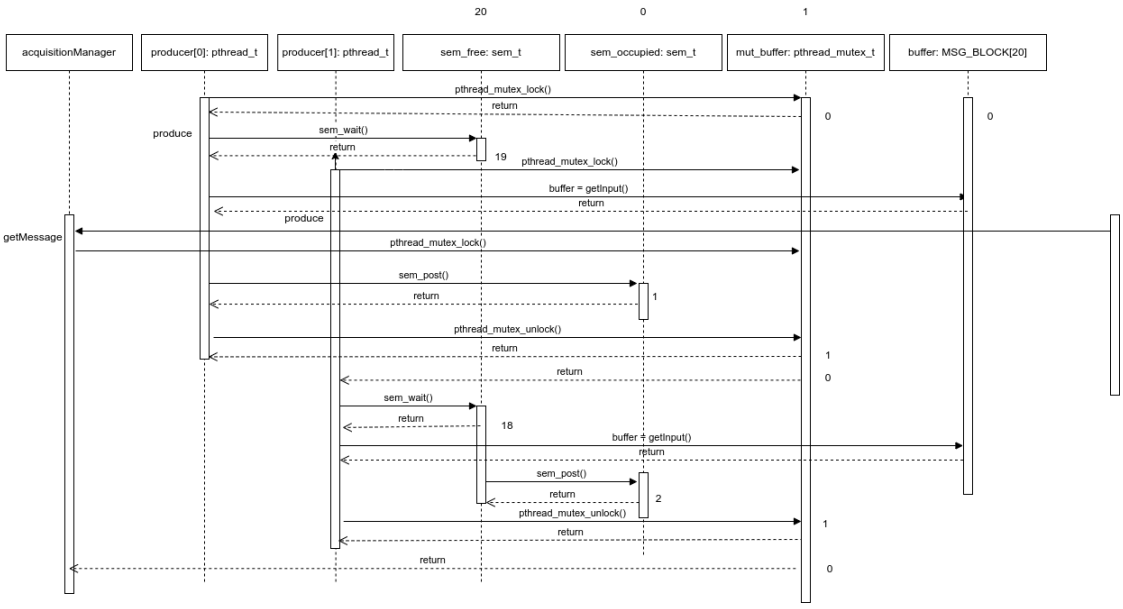


Figure 4: Diagramme séquentiel de `MultitaskingAccumulator`

```
[messageAdder] Creating thread
[acquisitionManager] Thread created for produce with id 59588
[messageAdder] Threads created
[displayManager] Creating thread
[displayManager] Threads created
[multitaskingAccumulator] Task initialization done.
[multitaskingAccumulator] Scheduling in progress...
[displayManager] Joining thread
[displayManager] Thread created for display with id 59590
[messageAdder] Thread created for sum with id 59589
Get message for input 1
Get message for input 1
[msg]....Sum done...
Get message for input 1
[OK       ] Checksum validated
Message
[1957747793 424238335 719885386 ... 705178736 1590079444 434248626 ]
[displayManager] Produced messages: 3, Consumed messages: 1, Messages left: 2
Get message for input 1
Get message for input 2
[acquisitionManager] 59588 termination
Get message for input 2
[acquisitionManager] 59587 termination
[msg]....Sum done...
Get message for input 2
[acquisitionManager] 59586 termination
[msg]....Sum done...
[OK       ] Checksum validated
```

```
Message
[881633576 3103498211 2683687007 ... 1818155256 2787612133 3497383497 ]
[displayManager]Produced messages: 7, Consumed messages: 3, Messages left: 4
[displayManager] 59590 termination
[displayManager]Threads joined
[messageAdder]Joining thread
Get message for input 2
[acquisitionManager] 59585 termination
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[messageAdder] 59589 termination
[messageAdder]Threads joined
[acquisitionManager]Joining threads
[acquisitionManager]Threads joined
[acquisitionManager]Semaphores cleaned
[multitaskingAccumulator]Threads terminated
```

7) Certaines variables du code sont considérées comme des variables volatiles car ce mot-clé précise au compilateur de n'effectuer aucune optimisation de typage, la variable en question pouvant être modifiée par un moyen extérieur au programme. Dans notre cas, des variables correspondant à des ressources partagées, comme `produceCount` doivent être considérées comme volatiles car elles peuvent être modifiées par différentes tâches.

8) À la différence des tâches, qui partagent le même espace mémoire, les processus sont soumis à un partitionnement spatial de la mémoire, ce qui assure que les variables d'un processus ne soient pas modifiées par un autre par inadvertance. Toutefois, les processus nécessitent plus d'espace mémoire et les échanges de données entre processus sont bien plus longs qu'entre tâches. Puisque chaque processus dispose de sa propre zone mémoire, il n'y a pas de concurrence sur les ressources, donc les stratégies de conception que nous avons utilisées ne sont pas applicables dans le cas des processus.

9) Afin de protéger la variable `produceCount` entre les tâches sans utiliser les APIs POSIX, il est possible de déclarer cette variable avec le mot-clé `_Atomic`, utilisable grâce au header `stdatomic.h`, et qui assure que les accès en lecture et en écriture à cette variable sont atomiques entre eux.

10) Les codes correspondants à cette implémentation sont joins avec ce rapport. Son exécution conduit à la sortie suivante :

```
[multitaskingAccumulator]Software initialization in progress...
[acquisitionManager]Synchronization initialization in progress...
[acquisitionManager]Creating semaphores
[acquisitionManager]Semaphores created
[acquisitionManager]Synchronization initialization done.
[acquisitionManager]Creating threads
[acquisitionManager]Thread created for produce with id 62501
[acquisitionManager]Thread created for produce with id 62502
```

```
[acquisitionManager]Thread created for produce with id 62503
[acquisitionManager]Threads created
[messageAdder]Creating thread
[acquisitionManager]Thread created for produce with id 62504
[messageAdder]Threads created
[displayManager]Creating thread
[messageAdder]Thread created for sum with id 62505
[displayManager]Threads created
[multitaskingAccumulator]Task initialization done.
[multitaskingAccumulator]Scheduling in progress...
[displayManager]Joining thread
[displayManager]Thread created for display with id 62506
Get message for input 1
Get message for input 1
[msg]....Sum done...
Get message for input 1
[OK      ] Checksum validated
Message
[1957747793 424238335 719885386 ... 705178736 1590079444 434248626 ]
[displayManager]Produced messages: 3, Consumed messages: 1, Messages left: 2
Get message for input 1
[msg]....Sum done...
Get message for input 2
[acquisitionManager] 62503 termination
Get message for input 2
[acquisitionManager] 62504 termination
Get message for input 2
[acquisitionManager] 62502 termination
[msg]....Sum done...
[OK      ] Checksum validated
Message
[881633576 3103498211 2683687007 ... 1818155256 2787612133 3497383497 ]
[displayManager]Produced messages: 7, Consumed messages: 3, Messages left: 4
[displayManager] 62506 termination
[displayManager]Threads joined
[messageAdder]Joining thread
Get message for input 2
[acquisitionManager] 62501 termination
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[messageAdder] 62505 termination
[messageAdder]Threads joined
[acquisitionManager]Joining threads
[acquisitionManager]Threads joined
```

```
[acquisitionManager]Semaphores cleaned
[multitaskingAccumulator]Threads terminated
```

11) La fonction `atomic_compare_exchange_weak( a, b, c )` effectue un test d'égalité atomique entre `a` et `b`, et affecte `c` à `a` si `a == b` et à `b` sinon. On peut utiliser cette fonction et une variable atomique `lock` pour définir une section critique que seul 1 thread à la fois sera capable d'exécuter. On rentre dans la section critique en itérant sur `atomic_compare_exchange_weak( &lock, &expected, 1 )`, puis en fixant `expected` à 0 à l'entrée de la section critique. Inversement, il suffit de fixer atomiquement `lock` à 0 pour en sortir. Tant qu'un thread est dans cette section, `lock` sera fixée à 1.

12) Les codes correspondants à cette implémentation sont joins avec ce rapport. Son exécution conduit à la sortie suivante :

```
[multitaskingAccumulator]Software initialization in progress...
[acquisitionManager]Synchronization initialization in progress...
[acquisitionManager]Creating semaphores
[acquisitionManager]Semaphores created
[acquisitionManager]Synchronization initialization done.
[acquisitionManager]Creating threads
[acquisitionManager]Thread created for produce with id 66435
[acquisitionManager]Thread created for produce with id 66437
[acquisitionManager]Thread created for produce with id 66436
[acquisitionManager]Threads created
[messageAdder]Creating thread
[acquisitionManager]Thread created for produce with id 66438
[messageAdder]Threads created
[displayManager]Creating thread
[messageAdder]Thread created for sum with id 66439
[displayManager]Threads created
[multitaskingAccumulator]Task initialization done.
[multitaskingAccumulator]Scheduling in progress...
[displayManager]Thread created for display with id 66440
[displayManager]Joining thread
Get message for input 1
Get message for input 1
[msg]....Sum done...
Get message for input 1
[OK       ] Checksum validated
Message
[1957747793 424238335 719885386 ... 705178736 1590079444 434248626 ]
[displayManager]Produced messages: 3, Consumed messages: 1, Messages left: 2
Get message for input 1
Get message for input 2
[acquisitionManager] 66436 termination
[msg]....Sum done...
Get message for input 2
[acquisitionManager] 66438 termination
```

```
Get message for input 2
[acquisitionManager] 66437 termination
[msg]....Sum done...
[OK       ] Checksum validated
Message
[3428251258 1826825043 1272358802 ... 1766909426 2776711070 2451013150 ]
[displayManager]Produced messages: 7, Consumed messages: 3, Messages left: 4
[displayManager] 66440 termination
[displayManager]Threads joined
[messageAdder]Joining thread
Get message for input 2
[acquisitionManager] 66435 termination
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[msg]....Sum done...
[messageAdder] 66439 termination
[messageAdder]Threads joined
[acquisitionManager]Joining threads
[acquisitionManager]Threads joined
[acquisitionManager]Semaphores cleaned
[multitaskingAccumulator]Threads terminated
```

13) On remarque que les solutions `acquisitionManagerPOSIX` et `acquisitionManagerTestAndSet` sont bien plus longues que `acquisitionAtomic`, et présentent notamment une importante variabilité dans leur temps d'exécution par rapport à la 3ème. Il est logique que la solution la plus rapide soit celle qui n'utilise que des variables atomiques, car la version avec la section critique (`acquisitionManagerTestAndSet`) nuit par définition au parallélisme des tâches, tandis que la version utilisant POSIX comporte 4 mutex, qui peuvent bloquer le flot d'exécution d'un thread qui attend de pouvoir se saisir d'un mutex.

14) Une approche dirigée par le temps est une approche synchrone car elle est cadencée par les battements d'une horloge.

15) La figure 5 correspond au diagramme temporel de `MultitaskingAccumulator` avec les 2 nouvelles exigences :

3 Second exercice

1) Les ressources partagées par les philosophes sont les fourchettes de la table.

2) Pour avoir un inter-blocage, il suffit que tous les philosophes passent à table en même temps, prennent leur fourchette de gauche, et attendent indéfiniment la fourchette à sa droite, laquelle est prise par l'un de ses concurrents.

3) Une solution pour éviter l'inter-blocage est de définir un philosophe dominant, les autres philosophes rendant

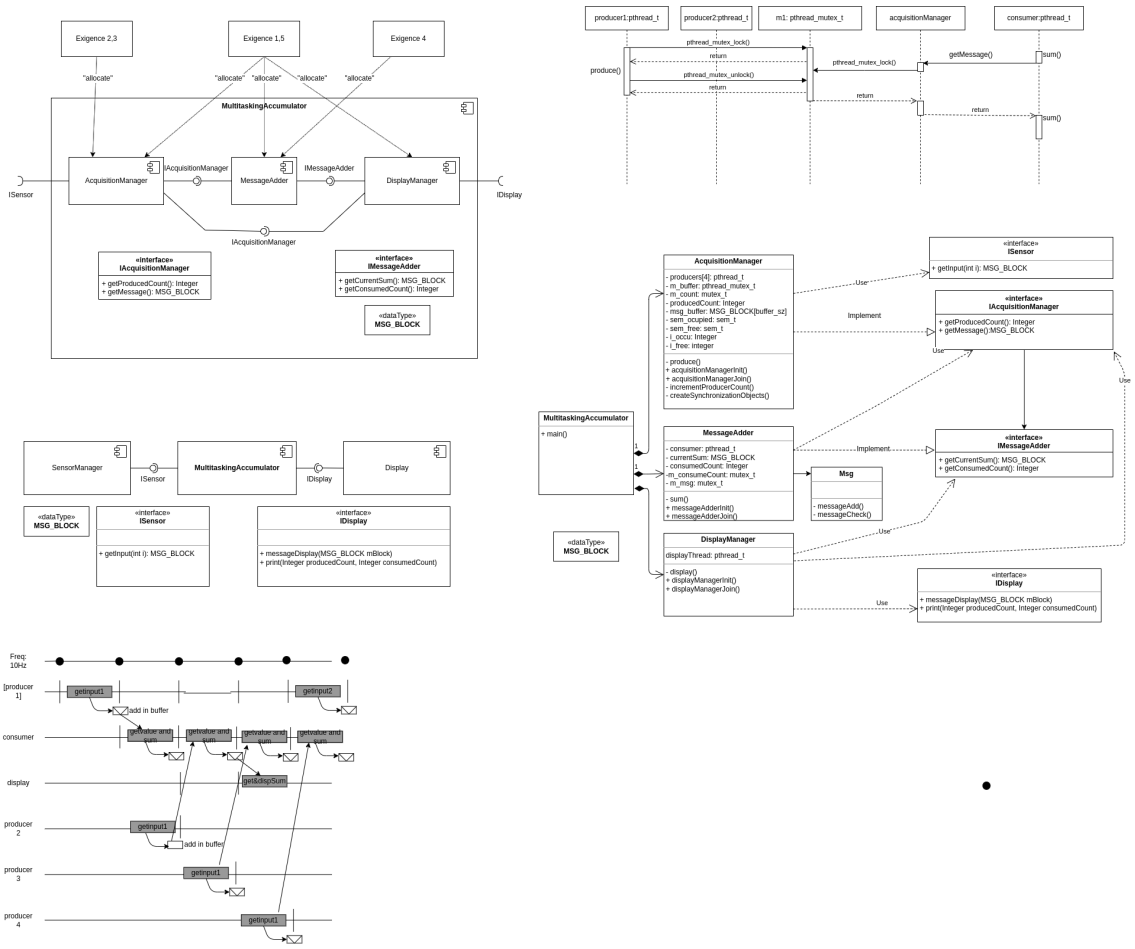


Figure 5: Diagramme temporel de `MultitaskingAccumulator`

régulièrement leur fourchette de gauche, ce qui finira par permettre au philosophe dominant de manger. Une fois qu'il aura terminé, il déposera sa fourchette, libérant ainsi cette ressource pour un autre philosophe, et ainsi de suite.